

Less Harness, More Signal: Efficient In-Harness RL for Autonomous Agents

Yufei Liu^{1*}, Yirong Zeng^{2*}, Yuxiang He, Yutai Hou³, Shen You², Jinhang Feng¹, Hao Cong⁴,
Yuxian Wang³, Wang Xu³, Wu Ning³, Xiao Ding², Qixun Zhang¹

¹Peking University, ²Harbin Institute of Technology, SCIR Lab,

³Huawei Technologies Co., Ltd, ⁴Tsinghua University

*Authors contributed equally, [†]Work done while interning at Huawei.

Abstract

In-harness reinforcement learning (RL), where each rollout executes within a live environment (e.g., OpenClaw), has become essential for training autonomous LLM-based agents. However, repurposing such harnesses for RL training introduces two critical bottlenecks: (1) environment execution dominates per-episode wall-clock time, severely underutilizing GPU resources; and (2) trajectory-level credit assignment in standard RL algorithms reinforces ineffective interactions alongside productive steps, leading to suboptimal policy convergence in autonomous tasks. We address both issues in a unified framework. On the systems side, we propose *ClawLoop*, a lightweight white-box training harness that strips non-essential product components while retaining a minimal five-element interaction loop. This design reduces environment overhead by $8.8\times$ and lifts GPU utilization from 14% to 49%. Building on this infrastructure, we propose *Asymmetric Advantage Masking* to resolving the credit misassignment problem in RL. It is a token-level correction mechanism that selectively suppresses spurious positive gradients on ineffective actions while preserving negative signals. Experiments across five benchmarks and model scales from 2B to 27B parameters show that our framework eliminates training collapse, improves success rate by up to +7.8 points over the base model, matches frontier proprietary models at 27B scale, and reduces inference-time token consumption by 32%.

Code — <https://github.com/zeng-yirong/clawloop>

Datasets — <https://huggingface.co/clawLoop>

1 Introduction

LLMs are evolving from passive text generators into proactive, tool-using problem solvers, i.e., *autonomous agents* that pursue complex goals over long, multi-turn horizons by iteratively planning and invoking external tools (Lopopolo 2026). Delivering this ability end to end requires an *execution harness*: low-level infrastructure that grounds the agent in a runnable environment through a sandboxed runtime, automated observability, and a dynamic feedback loop. Representative harnesses include OpenClaw (OpenClaw Contributors 2026), Hermes-Agent (Nous Research 2026), Claude Code (Anthropic 2025), and Codex (OpenAI 2025).

Unlike traditional tasks, Claw-style tasks operate over long-horizon trajectories involving multi-turn tool calling, interactive trial-and-error, and continuous environmental state

transitions. Training such agents has progressed from supervised fine-tuning on synthesized trajectories (Bai et al. 2026a; Xia et al. 2026) to on-policy RL executed directly inside the harness (Xu et al. 2026; Wang et al. 2026), where each rollout interleaves model generation with real environment feedback and is scored by an external verifier. This *in-harness RL* recipe is essential: only by closing the loop over live tools can the policy acquire multi-turn planning, error recovery, and state-tracking capabilities that static distillation cannot provide. However, repurposing product-grade harnesses as RL training environments introduces new bottlenecks (as shown in Fig. 1), rendering training both prohibitively slow and fundamentally unstable.

The first bottleneck is *environment overhead*. Product harnesses are engineered for end-user functionality rather than for high-throughput gradient estimation. We quantify this gap via a controlled comparison (§3.2): under a full OpenClaw harness, environment execution consumes 94% of per-episode wall-clock time (54.1 s), making in-harness rollouts $9.7\times$ slower than a tool-only baseline on the *same* task. Crucially, the bottleneck is not GPU-bound: GPU utilization collapses from 61% to 14% because the accelerator idles while the system waits on CPU/IO-bound sandbox interactions. Addressing this necessitates a training-oriented harness that strips product-level complexity while retaining the minimal interaction loop required for policy gradient computation.

The second bottleneck is *credit misassignment*. Standard on-policy RL algorithms (e.g., GRPO (Guo et al. 2025)) broadcasts a trajectory-level advantage to every generated token. In long-horizon tasks, a trajectory may succeed despite containing ineffective interactions, e.g., truncated turns and action errors. Because the sparse binary reward cannot distinguish productive steps from wasted ones, positive gradient reinforces both indiscriminately. We show empirically (§3.3) that the fraction of positive learning signal landing on ineffective tokens, ψ_+ , surges from 0.04 in early training to 0.20 after the performance peak, at which point the success rate *collapses*: the policy has learned to reproduce lucky-but-inefficient behavior.

We address both bottlenecks in a unified framework. On the systems side, we introduce *ClawLoop*, a lightweight, white-box training harness reconstructed from OpenClaw by removing all non-essential product components (e.g., gateway, plugin registries, background services) and retaining

a minimal five-element closed loop: task specification, environment state, tool execution, multi-turn interaction, and terminal verification. It exposes token-level log-probabilities and structured rollout records, enabling direct policy-gradient computation without parsing external APIs, and integrates as a thin environment interface into the VERL distributed RL system (Sheng et al. 2024). On the algorithmic side, we propose *Asymmetric Advantage Masking (AAM)*, a token-level correction that identifies ineffective interactions via four deterministic environment-side rules and suppresses their gradient contribution *only when the advantage is positive*. Negative advantages are preserved, so the policy continues to receive penalization signals for failed trajectories.

Experiments on five benchmarks, including three in-domain Claw-style tasks (PinchBench, ClawEval, ClawBench-Pro) and two out-of-domain tool-use settings (BFCL-v3, τ^2 -bench), demonstrate consistent gains across model scales from 2B to 27B parameters. On the efficiency side, ClawLoop achieves an $8.8\times$ reduction in per-episode environment time and lifts GPU utilization from 14% to 49%. On the training side, AAM eliminates the training collapse observed under standard GRPO, improving success rate by +5.1 points over GRPO and +7.8 points over the base model on Qwen3.5-9B. At the 27B scale, AAM-trained agents match frontier proprietary models (e.g., GPT-5.4) on in-domain harness tasks. At inference time, AAM further reduces per-episode token consumption by 32% on Qwen3.5-9B while maintaining competitive accuracy, confirming that correcting credit misassignment yields not only more accurate but also more efficient agent behavior.

2 Related Works

Agent Harness. Agent harnesses provide the execution infrastructure, integrating sandboxes, observability, and feedback loops, that mediates interactions between LLMs and external environments (Lopopolo 2026). Pioneered by domain-specific orchestrators like Claude Code (Anthropic 2025) and Codex (OpenAI 2025) in software engineering, harnesses have evolved into general-purpose frameworks such as OpenClaw (Steinberger 2026), NanoClaw (QwibitAI 2026), and Hermes-Agent (Nous Research 2026) to support open-ended tasks. To measure these capabilities, benchmarks within the Claw-style ecosystem, including PinchBench (Kilo AI Team 2026) and Claw-Eval (Ye et al. 2026), evaluate agents across coding, web interaction, and autonomous research workflows.

Claw-style Agent Training. Recent advances in Claw-style agent training can be categorized into two paradigms. The first paradigm relies on trajectory synthesis and SFT (Bai et al. 2026a; Li et al. 2026; Xia et al. 2026). For instance, ClawGym (Bai et al. 2026a) generates large-scale executable tasks and collects high-fidelity trajectories via black-box rollouts. The second paradigm employs On-Policy Distillation (OPD) to acquire long-horizon task-solving capabilities from teacher models (Wang et al. 2026; Yang et al. 2026; Bai et al. 2026b); e.g., OpenClaw-RL (Wang et al. 2026) optimizes policies online by extracting instructional signals from interactive environments. However, both approaches are fundamentally bottlenecked—either capped by static trajectory or teacher.

To break these limitations, in-harness RL pushes this performance ceiling (Yue et al. 2025; Guo et al. 2025). Pioneering works (Xu et al. 2026; Bai et al. 2026a), like Polar (Xu et al. 2026), treat the harness as a black box and enable on-policy RL training by intercepting interactions traces via proxy APIs. However, on-policy RL in complex harnesses suffers from prohibitively high computational cost and long training latency. In this paper, we address this critical bottleneck by proposing an algorithmically and computationally optimized framework for practical and lightweight on-policy RL training of Claw-style agents.

3 Preliminaries

We first formalize Group Relative Policy Optimization (GRPO (Guo et al. 2025)) in the context of agent trajectories, then examine the specific challenges arising from in-harness RL training.

3.1 RL for Autonomous Agents

Consider a agent that interacts with an environment and produces a token-level trajectory $\tau = (x_1, \dots, x_T)$. The policy is trained to maximize the expected return $\mathbb{E}_{\tau \sim \pi_\theta} [R(\tau)]$, where $R(\tau) \in \{0, 1\}$ is a sparse outcome reward, e.g., task success, returned by an external verifier. GRPO estimates advantages without a value model: for each prompt it samples a group of rollouts $\mathcal{G} = \{\tau_1, \dots, \tau_G\}$ and standardizes their returns. The resulting advantage is a trajectory-level quantity broadcast to all of its tokens,

$$\hat{A}_t = \hat{A}(\tau_i) = \frac{R(\tau_i) - \mu_{\mathcal{G}}}{\sigma_{\mathcal{G}}}, \quad (1)$$

where $\mu_{\mathcal{G}}, \sigma_{\mathcal{G}}$ are the mean and standard deviation of $\{R(\tau_j)\}_{j=1}^G$, so $\hat{A}_t$ is constant within τ_i . Restricting the clipped surrogate to policy-generated tokens gives the GRPO objective

$$\mathcal{J}_{\text{GRPO}}(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta_{\text{old}}}} \left[\frac{1}{T} \sum_{t=1}^T \ell_t(\theta) \right], \quad (2)$$

$$\ell_t(\theta) = \min \left(\rho_t(\theta) \hat{A}_t, \text{clip}(\rho_t(\theta), 1-\epsilon, 1+\epsilon) \hat{A}_t \right),$$

with importance ratio $\rho_t(\theta) = \pi_\theta(x_t \mid x_{<t}) / \pi_{\theta_{\text{old}}}(x_t \mid x_{<t})$.

3.2 The Cost of In-Harness Rollouts

To isolate the overhead of in-harness training, we conduct a controlled comparison on a single harness, OpenClaw (OpenClaw Contributors 2026), using the DAPO-Math-17k mathematical reasoning dataset (Feng et al. 2025). We train the same model with identical generation and optimization settings under two modes: (1) *in-harness*, where rollouts execute inside the full harness (e.g., sandboxed runtime, multi-tool orchestration, observability, and session state), following Bai et al. (2026a), and (2) *tool-only*, where the LLM is augmented with a single Python-interpreter tool and no full harness environment. Both modes execute tools in real runtime environments and differ exclusively in their execution infrastructure.

We conduct RL training on the Qwen3-8B model using 64 GPUs. Accounting per episode (with the per-step optimization

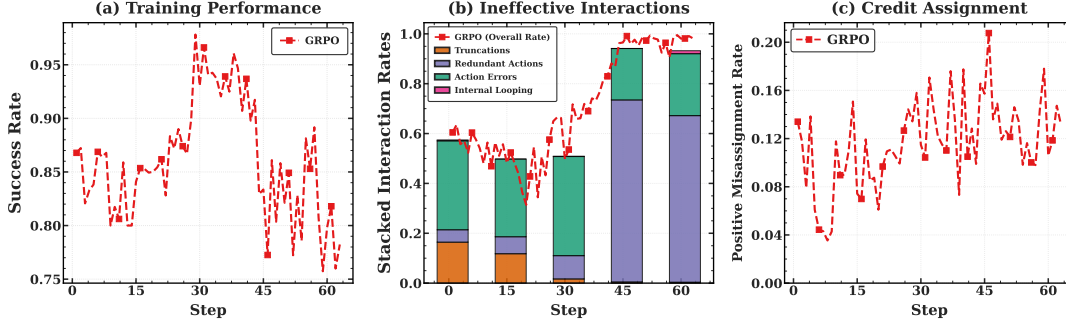


Figure 1: Training dynamics of GRPO in harness. (a) Success rate $\bar{p}(t)$ for GRPO; (b) Stacked ineffective-interaction rates $r_{\text{inv}}(t)$; (c) Positive-gradient misassignment $\psi_+(t)$.

time amortized over the global batch), the wall-clock time decomposes as:

$$T_{\text{ep}} = T_{\text{gen}} + T_{\text{env}} + T_{\text{train}}, \quad (3)$$

where T_{gen} and T_{env} denote the time spent on rollout generation and environment execution/feedback, respectively, and T_{train} represents the parameter optimization time.

As Fig. 2(a) shows, environment execution accounts for 94% of per-episode time under in-harness training ($T_{\text{env}} = 54.1$ s), rendering it $9.7\times$ slower than the tool-only mode on the *same* task ($T_{\text{env}} = 2.5$ s). Crucially, this gap is not explained by tool execution alone, the tool-only mode also runs the interpreter, but by the full harness stack that surrounds it. Furthermore, this bottleneck is not GPU-bound: GPU utilization drastically drops from 61% (tool-only) to 14% (in-harness) (Fig. 2(b)). This collapse occurs because generation is brief and batched, leaving GPUs idle while the system waits on the CPU/IO-bound environment interactions following each observation. Thus, the bottleneck is the environment, not the accelerator, and cannot be removed by adding GPUs.

Implication. Each rollout is both computationally costly and irreplaceable for sample efficiency. This motivates us to build a lightweight harness that strips away non-essential harness stack components from the agent loop, thereby reducing the time overhead of environment interactions.

3.3 Train Collapse in Long-Horizon Tasks

Collapse. We first train Qwen3.5-9B (Qwen Team 2026) on our lightweight harness with standard GRPO and track the success rate $\bar{p}(t)$ (Fig. 1a). The curve first climbs to a peak $p^* \approx 0.95$ at step $t^* \approx 30$ and then turns downward—we call this downturn a *collapse*. We mark collapse at the first step $t_c > t^*$ where $\bar{p}(t) \leq p^*$ holds for 3 consecutive evaluations. The verifier reward stays well-defined throughout, so the failure lies not in the objective but in how credit is assigned across a long rollout. Under GRPO this credit is spread evenly over every token, so we argue that when a long episode succeeds by luck, its ineffective steps are reinforced together with the useful ones.

Cause Analysis. To explain the post- t_c degradation, we qualitatively inspect the failing trajectories in $[t_c, t_{\text{end}}]$ and

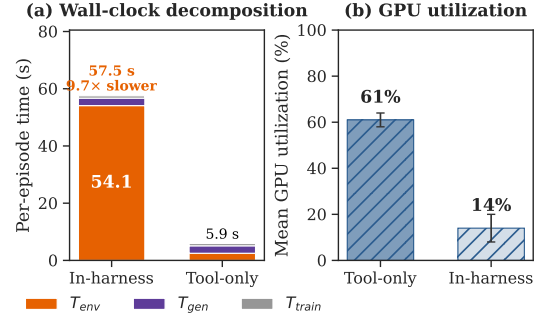


Figure 2: In-harness rollouts are dominated by environment execution. (a) Per-episode wall-clock decomposition: T_{env} accounts for 94% under in-harness training. (b) GPU utilization collapses from 61% (tool-only) to 14% (in-harness).

find that their wasted effort concentrates in four recurring patterns:

1. *Internal Looping*—a span of tokens repeats verbatim within a turn;
2. *Redundant Actions*—an action–observation pair repeats an earlier one, yielding no new state;
3. *Truncations*—a turn is cut off at T_{max} before a valid action completes;
4. *Action Errors*—a tool call returns a failure signal ($o_n = \text{ERROR}$).

These four categories are *induced* from the bad cases. They are not incidental: their per-step occurrence rate $r_{\text{inv}}(t)$ tracks the collapse in anti-phase (Fig. 1b), with a strong negative correlation (e.g., Pearson $\rho(\bar{p}, r_{\text{inv}}) = -0.83$ and Spearman rank correlation $\rho_S = -0.90$). The stacked decomposition shows the post- t_c surge is driven mainly by error and redundant calls, i.e. the very patterns that waste turns without advancing the task. This establishes, at the *behavioral* level, that collapse co-occurs with a rising of ineffective interactions.

Credit Misassignment. However, the behavioral correlation above does not yet explain *why* ineffective interactions grow. We therefore inspect the policy-gradient signal itself. For a generated token i in the rollouts used at training step t , let $w_i^+ = \mathbb{I}(\hat{A}_t > 0)$ denote the positive reinforcement

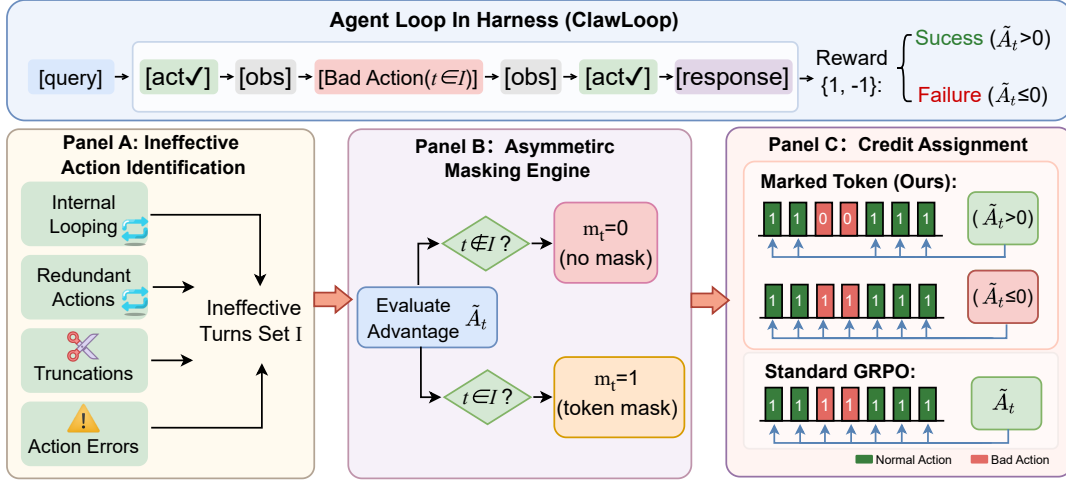


Figure 3: ClawLoop-based Asymmetric Advantage Masking for RL in-Harness. Asymmetric Advantage Masking (AAM) corrects credit misassignment by selectively suppressing positive gradients on ineffective tokens while preserving negative signals.

weight assigned by GRPO; the share of such mass falling on ineffective tokens is

$$\psi_+(t) = \frac{\sum_i w_i^+ \mathbb{I}(i \in \mathcal{I})}{\sum_i w_i^+}, \quad (4)$$

where $\mathcal{I}$ be the set of tokens within ineffective interactions. As shown in Fig. 1c, $\psi_+(t)$ remains low in early training (≈ 0.10) but rises sharply after the performance peak t^* , reaching ≈ 0.21 in the collapse region. As shown in Fig. 1c, $\psi_+(t)$ rapidly decreases from 0.12 to 0.04 within the first 10 steps. However, during steps 15–30, it surges quickly to 0.16 and reaches a peak of 0.20—precisely aligning with the rapid drop in success rate in Fig. 1a. Thus, in this region, a significant portion of the positive learning signal is misassigned to tokens that participate in ineffective interactions. Because GRPO broadcasts the trajectory-level advantage to every token, the ineffective tokens of a successful episode are reinforced alongside the effective ones. Therefore, we summarize that collapse is not merely accompanied by ineffective interactions; it is actively amplified by credit misassignment, confirming that successful trajectories themselves grow hollow.

Implication. Credit misassignment contributes to training collapse. To align the optimization signal with the actual contribution of each token, we propose the asymmetric advantage masking strategy introduced next.

4 Methodology

In this section, we present our framework for efficient and robust agent RL. We propose *ClawLoop*, a lightweight harness that minimizes environment overhead, and *Asymmetric Advantage Masking*, which corrects credit misassignment by suppressing spurious positive gradients (Fig. 3).

4.1 ClawLoop: A Lightweight Harness

From Product to Training. Product agent harness are typically optimized for user-facing functionality rather than

gradient estimation. Starting from OpenClaw (OpenClaw Contributors 2026), a product-oriented, open-source agent harness, we reconstruct a white-box, lightweight training harness, ClawLoop. Unlike black-box harnesses that expose only final textual outputs, ClawLoop exposes a unified rollout record of environment state (workspace file snapshots and tool execution logs), interaction history (per-step prompts and generated responses), and token-level log-probabilities of generated tokens. This design enables the RL system to compute policy gradients directly from the rollout, without parsing or reverse-engineering external product APIs.

ClawLoop follows a single design principle: *remove product-level complexity while retaining the minimal interaction loop required for agent RL*. Concretely, we remove the following components from the rollout critical path: persistent sessions and cross-episode state, long-term memory and user profiles, dynamic plugin registries and third-party service connections, background tasks and multi-agent orchestration, and authentication and permission control. These modules are essential for real-world deployment, but in RL rollouts they introduce hidden-state variance, uncontrollable external failure modes, and dynamically changing action spaces. The retained core is a minimal five-component closed loop comprising task specification, environment state, tool execution, multi-turn interaction, and terminal verification. Fig. 3 illustrates the resulting architecture and its data flow within the RL system.

Integration with RL training. ClawLoop is integrated as a thin environment interface into the VERL distributed RL system (Sheng et al. 2024), leaving the sampling scheduler and parameter-update logic unchanged. Each episode constitutes a finite-horizon MDP: the state $s_t = (q, o_{\leq t})$ concatenates the task instruction with observation history; transitions are deterministic given tool execution; reward is provided only at the terminal step by the verifier. This clean separation follows directly from the simplification above: with no shared memory, background services, or cross-episode dependencies, N trajectories can be executed independently on N workers.

Each worker maintains a single lightweight sandbox process, so tool errors in one trajectory do not contaminate others. As a result, per-episode environment-side overhead is reduced from 54.1 s (§3.2) to ~ 5 s (detailed in §5), and system throughput is primarily bounded by model inference throughput.

4.2 Asymmetric Advantage Masking

Ineffective Interaction Identification We instantiate the four patterns from §3 as deterministic, environment-side rules over the rollout. Let $\mathcal{T}_n$ be the indices of the tokens *generated* by the n -th assistant turn; prompt, observation, and padding tokens are excluded from all rules and from the loss. The union of flagged indices defines the global ineffective set $\mathcal{I} \subset \{1, \dots, T\}$:

1. **Internal Looping.** A repeated span $x_{t:t+k-1} = x_{t+k:t+2k-1}$ with $k \geq k_{\min}$ flags its redundant copy $\{t+k, \dots, t+2k-1\}$.
2. **Redundant Actions.** A turn n with $(a_n, o_n) = (a_{n'}, o_{n'})$ for some $n' < n$ flags $\mathcal{T}_n$.
3. **Truncations.** A turn cut off at the response budget $T_{\max}$ flags $\mathcal{T}_n$.
4. **Action Errors.** A turn whose observation is a failure signal ($o_n = \text{ERROR}$) flags $\mathcal{T}_n$.

Asymmetric Token-Level Masking Let $m_t^{\text{base}} \in \{0, 1\}$ be the standard masking vector (e.g., masking out padding tokens or prompt tokens). We introduce the asymmetric masking vector $m_t \in \{0, 1\}$, defined as:

$$m_t = m_t^{\text{base}} \cdot \left[1 - \mathbb{I}(t \in \mathcal{I} \wedge \hat{A}_t > 0) \right], \quad (5)$$

where $\mathbb{I}(\cdot)$ is the indicator function. The indicator fires only when an ineffective token carries a *positive* advantage. In GRPO the advantage is trajectory-level, $\hat{A}_t \equiv \hat{A}(\tau)$, so this is exactly the case where the episode was rewarded.

This asymmetric design ensures that we only *ignore* ineffective interactions when accidentally rewarded, but actively *punish* them when penalized, maintaining a complete and directional learning signal.

Policy Optimization with Masked Objectives We integrate the asymmetric mask m_t into the GRPO objective to form the Masked GRPO loss. The updated actor objective is computed as:

$$\mathcal{J}_{\text{Masked}}(\theta) = \mathbb{E}_{\tau} \left[\frac{1}{\sum_{t=1}^T m_t} \sum_{t=1}^T m_t \ell_t(\theta) \right], \quad (6)$$

$$\ell_t(\theta) = \min \left(\rho_t \hat{A}_t, \text{clip}(\rho_t, 1-\epsilon, 1+\epsilon) \hat{A}_t \right),$$

By re-weighting the loss with m_t , the optimization process dynamically excludes the tokens of ineffective interactions in successful trajectories from the policy gradient update.

Overall, this method acts as a fine-grained credit assignment corrector, effectively decoupling local behavioral quality from global trajectory outcomes in long-horizon agent training.

5 Experiments

5.1 Setup

Training data. Since no off-the-shelf corpus exists for claw-style agent training, we construct one by adapting the task-synthesis pipeline of ClawBenchPro (Herrieson 2026). The resulting dataset contains 7,000 instances, each defined as a triple $(q, \mathcal{W}_0, \mathcal{V})$: a natural-language task q , an initial workspace snapshot $\mathcal{W}_0$, and a deterministic terminal-state verifier $\mathcal{V}$. During rollout, the agent receives q and $\mathcal{W}_0$ as initial context; $\mathcal{V}$ is queried only at episode end to produce $R(\tau) \in \{-1, 1\}$.

Evaluation. We evaluate on two groups of benchmarks: The *in-domain* group comprises three claw-style agent benchmarks: PinchBench (Kilo AI Team 2026), ClawEval (Ye et al. 2026), and ClawBenchPro (Herrieson 2026), which share the workspace-and-tool interaction format of training. The *out-of-domain* group comprises two widely used multi-turn tool-use benchmarks, BFCL-v3 (Patil et al. 2025) and τ^2 -bench (Barres et al. 2025), whose tool schemas and dialogue protocols differ from training; strong performance here indicates that the learned agent behavior transfers beyond the synthetic distribution. For all benchmarks, we run three times and report both the average success rate.

Method	PinchBench	ClawEval	ClawBenchPro
<i>Proprietary</i>			
Kimi-2.5	54.60	66.60	73.00
Claude-4.6-Opus	69.90	80.60	92.30
GPT-5.4	75.70	78.30	93.70
MiniMax-m2.7	65.40	71.80	78.20
Nemotron3Super	42.20	41.70	46.50
<i>Open-source (Qwen3.5-9B)</i>			
Base	49.40 $\pm$ 0.4	65.20 $\pm$ 0.3	88.00 $\pm$ 0.2
+ GRPO	52.10 $\pm$ 0.3	67.50 $\pm$ 0.5	90.10 $\pm$ 0.2
+ AAM (ours)	57.20 $\pm$ 0.2	71.60 $\pm$ 0.4	93.60 $\pm$ 0.1
<i>Scale comparison (AAM)</i>			
Qwen3.5-2B	26.80 $\pm$ 0.5	29.10 $\pm$ 0.6	63.70 $\pm$ 0.4
+ AAM	41.80 $\pm$ 0.3	50.40 $\pm$ 0.4	73.70 $\pm$ 0.3
Qwen3.5-4B	37.10 $\pm$ 0.4	58.90 $\pm$ 0.3	87.60 $\pm$ 0.3
+ AAM	50.80 $\pm$ 0.2	72.40 $\pm$ 0.3	91.10 $\pm$ 0.2
Qwen3.5-27B	52.04 $\pm$ 0.3	72.30 $\pm$ 0.2	93.60 $\pm$ 0.1
+ AAM	57.54 $\pm$ 0.2	75.80 $\pm$ 0.3	95.80 $\pm$ 0.1

Table 1: Results on claw-style agent benchmarks. Reported values are mean $\pm$ std across 3 random seeds.

Implementation and Baselines. We fine-tune Qwen3.5 models (2B–27B) with RL, selecting this series as it has basic harness proficiency. We use GRPO within the VERL framework (Sheng et al. 2024), with ClawLoop as the interaction harness. Training is conducted on 40 GPUs with a rollout batch size of 64, update mini-batch size of 8, a 32K token context window, and up to 64 action turns per trajectory (details in SM). We compare against open-source Qwen3.5 baselines and top-tier proprietary models (GPT-5 (OpenAI 2025), Claude 4.6 (Anthropic 2024)), all evaluated under the standard OpenClaw harness for fair comparison on complex

Method	BFCL-v3	τ^2 -bench		
	Overall	Retail	Airline	Telecom
<i>Proprietary</i>				
Gemini-3-Pro	60.75	75.90	80.50	91.00
Claude Sonnet 4.5	61.37	72.40	72.00	84.90
Qwen3-235B-Think	42.75	71.90	58.60	47.30
Kimi-K2-Instruct	45.88	70.60	56.50	65.80
<i>Open-source (Qwen3.5-9B)</i>				
Base	44.00 $\pm$ 0.4	35.14 $\pm$ 0.3	32.00 $\pm$ 0.5	15.79 $\pm$ 0.2
+ GRPO	45.60 $\pm$ 0.3	36.18 $\pm$ 0.4	34.00 $\pm$ 0.3	17.93 $\pm$ 0.2
+ AAM (ours)	48.75 $\pm$ 0.2	38.23 $\pm$ 0.3	37.94 $\pm$ 0.2	22.15 $\pm$ 0.1
<i>Scale comparison (AAM)</i>				
Qwen3.5-2B	31.50 $\pm$ 0.5	25.44 $\pm$ 0.4	16.00 $\pm$ 0.6	35.96 $\pm$ 0.5
+ AAM	40.50 $\pm$ 0.3	31.30 $\pm$ 0.3	27.25 $\pm$ 0.4	48.01 $\pm$ 0.3
Qwen3.5-4B	44.25 $\pm$ 0.4	38.60 $\pm$ 0.3	30.00 $\pm$ 0.3	49.12 $\pm$ 0.4
+ AAM	48.50 $\pm$ 0.2	41.37 $\pm$ 0.2	35.31 $\pm$ 0.2	54.81 $\pm$ 0.2
Qwen3.5-27B	61.00 $\pm$ 0.3	64.91 $\pm$ 0.2	72.00 $\pm$ 0.3	73.68 $\pm$ 0.2
+ AAM	64.50 $\pm$ 0.1	67.19 $\pm$ 0.2	76.38 $\pm$ 0.1	78.37 $\pm$ 0.1

Table 2: Out-of-domain results on multi-turn tool-use benchmarks. Reported as mean $\pm$ std across 3 random seeds.

multi-turn tasks. At inference, we enable thinking mode and keep other parameters as defaults.

5.2 Main Results

On three claw-style benchmarks (Table 1), AAM consistently outperforms standard GRPO across every model scale. The gain is largest on PINCHBENCH (+**5.10** SR on Qwen3.5-9B), followed by CLAWVAL (+**4.10**) and CLAWBENCHPRO (+**3.60**). This ordering aligns with task horizon: PINCHBENCH contains the longest and most compositional episodes, where credit misassignment is most severe and suppressing reinforcement of ineffective tokens yields the largest return. Moreover, its success rate is comparable to that of frontier API-based LLMs with over 300B parameters. At 27B scale, AAM reaches **95.80%** SR on ClawBenchPro, narrowing the gap to GPT-5.4 (**93.70%**) from a **0.10** point difference to outperforming it by **2.10** points, suggesting that better credit assignment can partially substitute for scale.

On two tool-use benchmarks (Tables 2), AAM also transfers out-of-domain: it improves over GRPO by +**3.15** on BFCL-v3 and +**3.33** on τ^2 -bench, despite differing tool schemas and dialogue protocols. This transfer is notable because training data contains no BFCL or τ^2 -bench tasks; the gain therefore reflects a general improvement in multi-turn planning and error recovery rather than format memorization. We attribute this to AAM concentrating gradients on informative actions, which produces cleaner tool-call sequences that generalize across schemas. Accordingly, the gain is largest on longer τ^2 -bench domains (Airline and Telecom), mirroring the trend observed on claw-style benchmarks. Overall, correcting credit misassignment benefits not only in-distribution task completion but also cross-domain tool use.

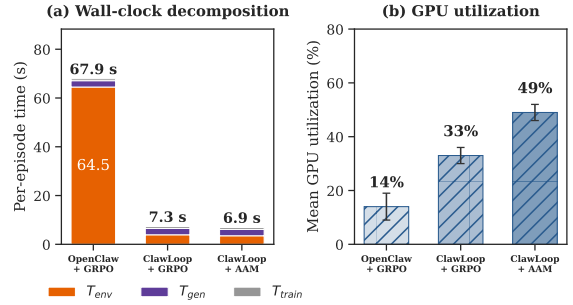


Figure 4: Training-side efficiency. (a) Per-episode wall-clock time decomposition. (b) Mean GPU utilization. ClawLoop drastically reduces environment overhead, while AAM further improves efficiency by minimizing redundant interactions.

5.3 Efficiency Analysis

Training-side efficiency. We compare three training configurations under identical settings: OpenClaw in-harness GRPO, ClawLoop with GRPO, and ClawLoop with AAM. As shown in Fig. 4(a), the product-level harness remains dominated by environment execution, costing 64.5 s per episode. ClawLoop reduces this to 7.3 s, achieving a 8.8 $\times$ speedup by stripping non-essential components from the rollout path. Notably, AAM further cuts per-episode time to 6.9 s, as suppressing ineffective interactions shortens average trajectory length. This reduction in wasted computation translates to higher efficiency: mean GPU utilization increases from 14% (OpenClaw) to 33% (ClawLoop + GRPO), and further to 49% with AAM (Fig. 4(b)). These results demonstrate that eliminating ineffective interactions is critical for improving GPU utilization, as it minimizes idle time caused by redundant environment feedback and unnecessary token generation.

Inference-side efficiency. We further examine whether the efficiency gain persists at test time. Fig. 6 plots success rate against average token consumption per episode. For each model family, we show results from 2B to 27B parameters. The base Qwen3.5 models improve as scale increases, but still consume a large number of tokens while achieving moderate success rates, indicating that many generated steps do not contribute to task completion. After training with ClawLoop and AAM, our models at every scale move toward the lower-right region: they achieve higher success rates while using fewer tokens. This gap widens at larger scales: Qwen3.5-27B with AAM reaches **66.4%** SR with only **5.7K** tokens per episode, approaching GPT-5 (**67.2%** SR) while using substantially fewer tokens. The result suggests that AAM does not merely improve accuracy; it produces more efficient agent behavior by reducing ineffective interaction steps.

5.4 Training Dynamics

Here, we compare the training dynamics of standard GRPO and AAM on Qwen3.5-9B under the same setting as in Section §3.3. Fig. 5 reports three curves: (a) success rate $\bar{p}(t)$, (b) ineffective-interaction rate $r_{inv}(t)$, and (c) positive-gradient misassignment $\psi_+(t)$.

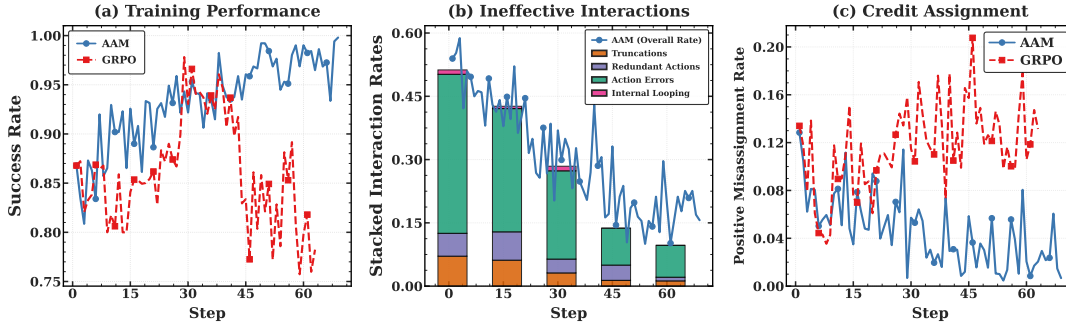


Figure 5: Training dynamics of GRPO vs. AAM on Qwen3.5-9B. (a) Success rate $\bar{p}(t)$ for GRPO and AAM; (b) Stacked ineffective-interaction rates $r_{\text{inv}}(t)$; (c) Positive-gradient misassignment $\psi_+(t)$ (occurred in GRPO vs. prevented by AAM).

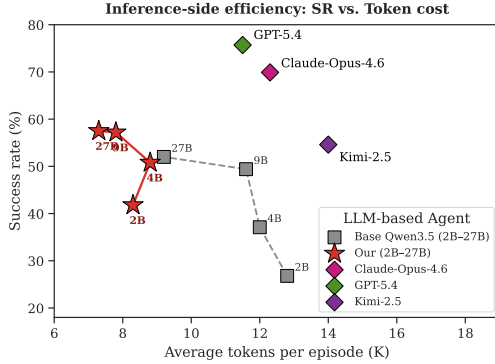


Figure 6: Inference-side efficiency. Success rate as a function of average token consumption per episode.

AAM substantially alters the collapse behavior during training. Its success rate continues to improve after the GRPO collapse point and stabilizes at a higher plateau, with no sustained post-peak degradation. Meanwhile, $r_{\text{inv}}(t)$ remains low, and $\psi_+(t)$ stays near its early-training level, indicating that positive reinforcement no longer accumulates on ineffective tokens. These curves suggest that GRPO collapse arises because successful but inefficient episodes reinforce ineffective tokens along with productive ones. AAM breaks this feedback loop by masking positive advantage on ineffective tokens, concentrating learning on actions that advance the task, yielding more stable training and a higher success rate.

5.5 Ablation Study

To isolate AAM’s gains, we conduct ablations on Qwen3.5-9B and report average PinchBench success rate in Table 3.

Each ineffective-pattern mask contributes. Disabling any single pattern mask reduces performance. The largest drops come from removing masks for redundant actions and action errors (−4.10 and −3.80 SR), indicating that these tokens are especially harmful when they receive positive reinforcement in otherwise successful trajectories. The masks for internal looping and truncation produce smaller but consistent effects (−0.30 and −0.40 SR). This result is broadly consistent with the training-dynamics analysis in §3.3: redundant interactions

Variant	Avg. SR	Δ
AAM (full)	57.20	—
w/o Looping	56.90	−0.30
w/o Redundant	53.10	−4.10
w/o Truncation	56.80	−0.40
w/o Error	53.40	−3.80
Symmetric masking	50.90	−6.30
Random masking	51.70	−5.50

Table 3: Ablation of AAM on Qwen3.5-9B. Avg. SR (%) is the average in-domain success rate. Δ denotes the change.

directly waste valid turns, while action errors provide explicit failure signals that should not be positively reinforced.

Asymmetric masking is essential. Symmetric masking, which zeros gradients on ineffective tokens regardless of advantage sign, underperforms full AAM by 6.30 SR. This shows that simply removing gradient signal from ineffective tokens is insufficient. AAM suppresses only *positive* reinforcement on ineffective tokens while preserving *negative* advantage, allowing the policy to learn avoidance rather than merely forgetting the corresponding behavior.

The gain is not due to random gradient sparsification. Random masking, which zeros the same fraction of tokens uniformly at random, trails full AAM by 5.50 SR and does not recover AAM’s improvement. This rules out generic regularization or reduced gradient mass as the main source of gain: the benefit depends on identifying ineffective tokens and applying advantage-asymmetric masking to them.

6 Conclusion

We identify environment overhead and credit misassignment as two bottlenecks in in-harness RL, addressing them through ClawLoop and Asymmetric Advantage Masking. ClawLoop strips product complexity for faster environment execution, while AAM corrects token-level credit assignment, eliminating training collapse. Experiments show consistent gains in training stability and inference efficiency. These results show that practical in-harness RL requires co-designing lightweight infrastructure and fine-grained optimization signals.

References

- Anthropic. 2024. Introducing Claude Opus 4.6. <https://www.anthropic.com/news/claude-opus-4-6>. Accessed: 2026-05-04.
- Anthropic. 2025. Claude Code. <https://www.anthropic.com/product/claude-code>. Accessed: 2026-05-02.
- Anthropic. 2025. Claude Code: An agentic coding tool for the terminal. <https://github.com/anthropics/claude-code>. Accessed: 2026-06-29.
- Bai, F.; Song, H.; Sun, S.; Cheng, D.; Yang, Y.; Hao, C.; Li, R.; Chang, F.; Wei, Y.; Tao, R.; et al. 2026a. ClawGym: A scalable framework for building effective claw agents. *arXiv preprint arXiv:2604.26904*.
- Bai, L.; Cao, Z.; Chen, Y.; Cui, Z.; Du, S.; Fan, Y.; Feng, S.; Guo, Z.; He, H.; He, L.; et al. 2026b. Scaling the Horizon, Not the Parameters: Reaching Trillion-Parameter Performance with a 35B Agent. *arXiv preprint arXiv:2606.30616*.
- Barres, V.; Dong, H.; Ray, S.; Si, X.; and Narasimhan, K. 2025. tau2-Bench: Evaluating Conversational Agents in a Dual-Control Environment. *arXiv preprint arXiv:2506.07982*.
- Feng, J.; Huang, S.; Qu, X.; Zhang, G.; Qin, Y.; Zhong, B.; Jiang, C.; Chi, J.; and Zhong, W. 2025. Retool: Reinforcement learning for strategic tool use in llms. *arXiv preprint arXiv:2504.11536*.
- Guo, D.; Yang, D.; Zhang, H.; Song, J.; Zhang, R.; Xu, R.; Zhu, Q.; Ma, S.; Wang, P.; Bi, X.; et al. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*.
- Herrieson. 2026. ClawBenchPro. <https://github.com/Herrieson/nanoclaw>. Accessed: 2026-07-26.
- Kilo AI Team. 2026. Pinchbench: Benchmarking system for evaluating LLM models as OpenClaw agents. <https://github.com/pinchbench/skill>. GitHub repository. Accessed: 2026-05-02.
- Li, X.; Li, M.; Stoica, I.; Hsieh, C.-J.; and Zhou, T. 2026. Clawenvkit: Automatic environment generation for claw-like agents. *arXiv preprint arXiv:2604.18543*.
- Lopopolo, R. 2026. Harness engineering: leveraging codex in an agent-first world. *OpenAI engineering note*.
- Nous Research. 2026. Hermes Agent: The agent that grows with you. <https://github.com/nousresearch/hermes-agent>. GitHub repository. Accessed: 2026-05-02.
- OpenAI. 2025. Introducing GPT-5.
- OpenAI. 2025. OpenAI Codex. <https://openai.com/codex/>. Accessed: 2026-05-02.
- OpenClaw Contributors. 2026. OpenClaw: An open-source framework and repository. <https://github.com/openclaw/openclaw>. Accessed: 2026-06-29.
- Patil, S. G.; Mao, H.; Yan, F.; Ji, C. C.-J.; Suresh, V.; Stoica, I.; and Gonzalez, J. E. 2025. The Berkeley Function Calling Leaderboard (BFCL): From Tool Use to Agentic Evaluation of Large Language Models. In *Forty-second International Conference on Machine Learning*.
- Qwen Team. 2026. Qwen3.5: Towards Native Multimodal Agents.
- QwibitAI. 2026. NanoClaw. <https://github.com/qwibitai/nanoclaw>.
- Sheng, G.; Zhang, C.; Ye, Z.; Wu, X.; Zhang, W.; Zhang, R.; Peng, Y.; Lin, H.; and Wu, C. 2024. HybridFlow: A Flexible and Efficient RLHF Framework. *arXiv preprint arXiv:2409.19256*.
- Steinberger, P. 2026. OpenClaw: Your own personal AI assistant. <https://github.com/openclaw/openclaw>. GitHub repository. Accessed: 2026-05-02.
- Wang, Y.; Chen, X.; Jin, X.; Wang, M.; and Yang, L. 2026. Openclaw-rl: Train any agent simply by talking. *arXiv preprint arXiv:2603.10165*.
- Xia, P.; Chen, J.; Yang, X.; Tu, H.; Liu, J.; Xiong, K.; Han, S.; Qiu, S.; Ji, H.; Zhou, Y.; et al. 2026. MetaClaw: Just Talk—An Agent That Meta-Learns and Evolves in the Wild. *arXiv preprint arXiv:2603.17187*.
- Xu, B.; Zhang, H.; Zhang, S.; Han, S.; Liu, M.; Hu, J.; Diao, S.; Jin, Z.; Zou, Y.; Demoret, M.; et al. 2026. Polar: Agentic RL on Any Harness at Scale. *arXiv preprint arXiv:2605.24220*.
- Yang, Z.; Liu, Z.; Chen, Y.; Dai, W.; Wang, B.; Lin, S.-C.; Lee, C.; Chen, Y.; Jiang, D.; He, J.; et al. 2026. Nemotron-cascade 2: Post-training llms with cascade rl and multi-domain on-policy distillation. *arXiv preprint arXiv:2603.19220*.
- Ye, B.; Li, R.; Yang, Q.; Liu, Y.; Yao, L.; Lv, H.; Xie, Z.; An, C.; Li, L.; Kong, L.; et al. 2026. Claw-Eval: Toward Trustworthy Evaluation of Autonomous Agents. *arXiv preprint arXiv:2604.06132*.
- Yue, Y.; Chen, Z.; Lu, R.; Zhao, A.; Wang, Z.; Yue, Y.; Song, S.; and Huang, G. 2025. Does Reinforcement Learning Really Incentivize Reasoning Capacity in LLMs Beyond the Base Model? In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.

A Results on 2B and 4B Models

We compare the base model, standard GRPO, and AAM using the same three Claw-style benchmarks as the main paper.

Method	PinchBench	ClawEval	ClawBenchPro
<i>Qwen3.5-2B</i>			
Base	26.80 $\pm$ 0.5	29.10 $\pm$ 0.6	63.70 $\pm$ 0.4
+ GRPO	34.10 $\pm$ 0.4	39.60 $\pm$ 0.5	68.50 $\pm$ 0.4
+ AAM (ours)	41.80 $\pm$ 0.3	50.40 $\pm$ 0.4	73.70 $\pm$ 0.3
<i>Qwen3.5-4B</i>			
Base	37.10 $\pm$ 0.4	58.90 $\pm$ 0.3	87.60 $\pm$ 0.3
+ GRPO	44.20 $\pm$ 0.3	65.10 $\pm$ 0.4	89.30 $\pm$ 0.3
+ AAM (ours)	50.80 $\pm$ 0.2	72.40 $\pm$ 0.3	91.10 $\pm$ 0.2

Table 4: Results on Claw-style agent benchmarks. Values are mean $\pm$ standard deviation across three random seeds.

B Training Configuration

The main optimization and sampling settings are summarized below.

Hyperparameter	Value
RL algorithm	GRPO
Actor learning rate	6e-7 (constant)
Adam optimizer (β_1, β_2)	(0.9, 0.95)
Weight decay	0.01
Global rollout batch size	64
GRPO group size (N)	8
PPO mini-batch size	16
PPO clip range (ϵ)	0.2
Maximum prompt length	8,192
Maximum response length	22,768
Maximum sequence length	30,960 ($\approx$ 31K)
Maximum turns	70
Sampling temperature	0.6
Sampling top- p	0.95
Sampling top- k	20
KL coefficient (β)	0.001 (low_var_kl)

Table 5: Training hyperparameters and sampling configuration.

C Training Data Example

Each synthesized task contains an instruction, an environment builder, and a workspace verifier. Below is a representative training example from data_000254.

Instruction (prompts.md)

Look, I need a collections priority report by end of week. I dumped everything into the workspace: billing invoices, accounting payment ledger, and client accounts. Note that invoice dates use mixed formats.

Today is October 20, 2024. Figure out which wholesale clients have unpaid invoices more than 60 days old (strictly past due; paid or cancelled invoices do not count).

Group overdue amounts by client, sort them in descending order, and write the output to 'collections/priority_list.json'. For each client, include their name, total overdue balance, and specific overdue invoice numbers.

Environment Builder (env_builder.py)

```
1 import os
2
3 def build_env():
4     os.makedirs("billing", exist_ok=True)
5     os.makedirs("payments", exist_ok=True)
6     os.makedirs("clients", exist_ok=True)
7     os.makedirs("collections", exist_ok=True)
```

```
8
9 # Invoices with mixed date formats & statuses
10 invoice_data = ("inv_id,client,date_issued,
11                amount,status\n"
12                "INV-001,FreshMart
13                ,07/15/2024,2400.00,PAID\n"
14                "INV-002,Green Valley Grocers
15                ,2024-07-20,5800.00,OPEN\n"
16                "INV-004,Boise Bulk Foods
17                ,2024-08-10,8500.00,OPEN\n"
18                "INV-005,FreshMart
19                ,08/15/2024,3100.00,OPEN\n"
20                "INV-011,Snake River Co-op
21                ,2024-07-25,3000.00,CANCELLED
22                \n")
23
24 with open("billing/invoices_raw.csv", "w") as f:
25     f.write(invoice_data)
26
27 payment_log = ("[2024-08-10] PAYMENT RECEIVED -
28                FreshMart - ref:INV-001 - $2,400.00\n"
29                "[2024-09-18] NOTE: INV-011
30                cancelled per client request\n"
31                ")
32
33 with open("payments/ledger_2024.log", "w") as f:
34     f.write(payment_log)
35
36 client_accounts = ("client_name\taccount_id\
37                   tterms\n"
38                   "FreshMart\tACC-1001\tNet 30\n"
39                   "Boise Bulk Foods\tACC-1004\tNet
40                   60\n")
41
42 with open("clients/accounts.tsv", "w") as f:
43     f.write(client_accounts)
44
45 if __name__ == "__main__":
46     build_env()
```

Workspace Verifier (workspace_verifier.py)

```
1 import os, sys, json, httpx
2 from pathlib import Path
3 from openai import OpenAI
4
5 MOCK_API_KEY = os.environ.get("MOCK_API_KEY", "
6 dummy_key")
7 MOCK_API_BASE = os.environ.get("MOCK_API_BASE", "
8 http://localhost/v1")
9 MOCK_MODEL_NAME = os.environ.get("MOCK_MODEL_NAME
10 ", "gpt-5.4")
11
12 client = OpenAI(api_key=MOCK_API_KEY, base_url=
13 MOCK_API_BASE, http_client=httpx.Client(
14 verify=False))
15
16 def llm_judge(prompt, file_content):
17     try:
```

```

13     res = client.chat.completions.create(
14         model=MOCK_MODEL_NAME,
15         messages=[{"role": "system", "content":
16             "Answer ONLY with 'YES' or 'NO'."},
17             {"role": "user", "content": f"{
18                 prompt}\n\n[Content]:\n{
19                     file_content}"},
20             ],
21         temperature=0
22     )
23     return "yes" in res.choices[0].message.
24         content.strip().lower()
25 except Exception:
26     return False
27
28 def main():
29     workspace = sys.argv[1] if len(sys.argv) > 1
30     else "."
31     output_path = Path(workspace) / "collections"
32     / "priority_list.json"
33     score_details, total_score = [], 0
34
35     if not output_path.exists():
36         score_details.append({"item": "File
37             Existence", "score": 0, "max_score":
38             10, "passed": False})
39         return
40
41     score_details.append({"item": "File Existence",
42         "score": 10, "max_score": 10, "passed":
43         True})
44     total_score += 10
45     content = output_path.read_text(encoding="utf
46         -8", errors="replace")
47
48     # Target Evaluations
49     checks = [
50         ("Verify Boise Bulk Foods overdue ($8,500)",
51             "Does content include Boise Bulk
52             Foods with $8,500 overdue?", 30),
53         ("Verify Descending Order", "Are customers
54             sorted in descending order by amount?",
55             30),
56         ("Exclude Cancelled INV-011", "Does content
57             EXCLUDE invoice INV-011?", 30)
58     ]
59
60     for name, prompt, pts in checks:
61         passed = llm_judge(prompt, content)
62         score_details.append({"item": name, "score":
63             pts if passed else 0, "max_score":
64             pts, "passed": passed})
65         if passed: total_score += pts
66
67     with open(os.path.join(workspace, "
68         workplace_score.json"), "w") as f:
69         json.dump({"total_score": total_score, "
70             details": score_details}, f, indent=2)
71
72 if __name__ == "__main__":
73     main()

```

D Data-Synthesis Prompt

The following prompt is used to generate the three files of a training task.

You are a top-tier AI Agent Evaluation Architect, Senior Screenwriter, and Full-Stack Engineer.

Your goal: Using inspiration from a given PERSONA seed, create a high-quality, highly discriminative nanoclaw Agent evaluation task from scratch for task ID {task_id}.

[Single-Pass Principle]

You MUST generate all four task components in a single response to maintain perfect consistency across the business logic, hidden ground truth, user prompts, sandbox data, and evaluation criteria.

[Consistency Constraints]

All 4 output files must align with the exact same ground truth:

- tasks/prompts/{task_id}.md: The business task given to the Agent, solvable purely from data generated by env_builder.py.
- scripts/{task_id}/verify_workplace.py: Evaluates only what was requested in the User Prompt.

[Output Format Requirements]

You must output exactly 4 files in order, enclosed in code blocks:

1. tasks/{task_id}.yaml
2. tasks/prompts/{task_id}.md
3. tasks/{task_id}/env_builder.py
4. scripts/{task_id}/verify_workplace.py

[Core Design Guidelines]

- User Prompts must omit step-by-step solutions or explicit ground-truth values.
- Environment Setup must generate realistic noise and non-standard data formats.
- Workplace Verifier combines path checks with LLM single-item YES/NO fuzzy evaluations.

E Examples of Ineffective Interactions

The four rules identify local interaction patterns that should not receive positive reinforcement merely because the full trajectory succeeds. Table 6 summarizes these patterns alongside their target masking span.

E.1 Concrete Trajectory Traces

Below are simplified execution traces highlighting where the mask is applied for each ineffective interaction pattern.

Pattern	Concrete example	Masked span
Internal looping	Thought block loops identical reasoning phrases in a single assistant turn.	Repeated thought span
Redundant action	<code>read_file(logs.csv)</code> called multiple times with identical observations.	Duplicate action turns
Truncation	Output exceeds $T_{\max}$ mid-JSON structure during file writing.	Incomplete assistant turn
Action error	<code>bash(cat missing.txt)</code> fails with non-zero exit code.	Error-producing turn

Table 6: Examples of the four ineffective-interaction patterns.

Pattern 1: Action Error (Execution Failure)

```
[Assistant Turn]
I need to inspect the district maps file before
generating the report.
<tool_call>
<function=bash><parameter=command>cat
    district_maps/lookup.txt</parameter></
    function>
</tool_call>

[Tool Output] (Error)
Error: bash exited with code 1
cat: district_maps/lookup.txt: No such file or
    directory
```

Pattern 2: Redundant Action (Repeated Identical Inspection)

```
[Turn 1 - Assistant]
I will inspect the signups CSV file.
<tool_call><function=bash><parameter=command>cat
    registrations/signups.csv</parameter></
    function></tool_call>

[Turn 1 - Tool Output]
Name, Status
James, Paid

[Turn 2 - Assistant] (Redundant Action Identified)
I need to inspect the signups CSV file again to
    ensure I didn't miss anything.
<tool_call><function=bash><parameter=command>cat
    registrations/signups.csv</parameter></
    function></tool_call>
```

Pattern 3: Truncation (Cutoff at $T_{\max}$)

```
[Assistant Turn]
Generating the full JSON configuration output...
<tool_call>
<function=write_file>
<parameter=path>output/plan.json</parameter>
<parameter=content>
{ "orders": [ { "id": 101, "val": 200}, { "id": 102,
    [TRUNCATED HERE AT
T_MAX]
```

Pattern 4: Internal Looping (Reasoning Loop in Single Turn)

```
[Assistant Thought]
I need to extract the API response times from the
    logs.

[Loop Block 1]
Wait, the annotation mentions background noise
    drowning out the API latency metrics...

[Loop Block 2 - Identical Repetition]
Wait, the annotation mentions background noise
    drowning out the API latency metrics...
```

F Agent Tools

The agent operates on an isolated workspace through the following nine tools.

Tool	Description
<code>list_dir</code>	List files and directories in the workspace.
<code>read_file</code>	Read a UTF-8 text file.
<code>write_file</code>	Create or overwrite a text file.
<code>edit_file</code>	Replace exact text in an existing file.
<code>apply_patch</code>	Apply multiple exact text replacements.
<code>grep</code>	Search workspace files with a regular expression.
<code>find</code>	Find workspace files using a glob pattern.
<code>mkdir</code>	Create a directory in the workspace.
<code>bash</code>	Run restricted commands or local scripts.

Table 7: Tools provided by the agent framework.

G Limitations

While our framework demonstrates strong empirical performance and efficiency gains, we acknowledge two main limitations that offer promising directions for future work.

First, our evaluation and experiments are primarily conducted on the Qwen-3.5 series (2B, 4B, 9B, and 27B). This model choice was intentional, as earlier Qwen3 variants exhibited weak baseline tool-use capabilities, making it difficult to disentangle fundamental tool-calling proficiency from

broader agentic reasoning improvements during RL training. However, we recognize that relying on a single model family limits the breadth of our findings. Extending our training framework and evaluating its generalizability across more diverse model architectures (e.g., Llama, Mistral) remains an important next step.

Second, the four invalid interaction modes targeted by Asymmetric Advantage Masking (AAM) do not form an exhaustive taxonomy of all potential agent failures. We prioritized these four specific patterns because they are easily detectable, unequivocally incorrect across contexts, and occurred with high frequency during our preliminary trajectory analysis. While addressing these fundamental patterns proved sufficient to eliminate training collapse, other subtler failure modes likely exist. Discovering and formulating masking mechanisms for additional failure types will require fine-grained empirical studies on broader datasets and diverse model families in future research.

H Ethical Statement

This work adheres to established ethics standards in AI research.

First, on data safety and environmental impact, our experiments rely solely on publicly available benchmarks and datasets. All benchmarks were checked to ensure they contain no personal identifiable information (PII), offensive content, or copyrighted code under restricted licenses. While training autonomous agents with live tool-use capabilities inherently carries risk if deployed in unconstrained environments, all agent executions in this work were strictly confined within isolated, sandboxed container environments (ClawLoop) without external network side-effects or persistent system privileges.

Second, regarding environmental efficiency, our proposed ClawLoop harness reduces per-episode environment execution overhead by $8.8\times$ and AAM optimizes inference token consumption by 32%. These optimizations significantly lower the computational energy footprint required for RL training and deployment of large language model agents.

I Generative AI Statement

In the preparation of this manuscript, the authors utilized Large Language Models strictly as writing and assistance tools for grammar correction, language polishing, and code formatting of the text. All conceptual formulations, algorithmic designs, experimental setups, result analyses, and manuscript final reviews were entirely conducted and verified by the authors. The authors take full responsibility for the accuracy and integrity of the content published in this paper.